

ER Modeling — Study Notes

Compiled from our tutoring session. Sections marked (NEW) are supplementary material not yet covered in our chat — added per your request for anything missed or important.

Phase 1: Core Concepts & Vocabulary

The Core Idea

Before building actual database tables, you first sketch out what things your system needs to track, and how those things relate to each other. That sketch is an ER (Entity-Relationship) model — a blueprint drawn before writing any SQL.

Example: A library system needs to track books and members, and capture that a member borrows a book.

Vocabulary

Term	Definition	Example
Entity	A distinct, independently-identifiable “thing” you store data about	Book, Member, Employee
Attribute	A property that describes an entity	Book has <code>title</code> , <code>ISBN</code> , <code>publication_year</code>
Relationship	An association between entities	Member borrows Book
Key Attribute	Attribute(s) that uniquely identify an entity instance	ISBN uniquely identifies a Book
Composite Attribute	An attribute divisible into meaningful sub-parts	Address -> <code>street</code> , <code>city</code> , <code>zip_code</code>
Multivalued Attribute	An attribute that can hold multiple values for one instance	A Book can have several <code>genres</code>
Derived Attribute	An attribute computable from other attributes, not stored directly	<code>age</code> derived from <code>date_of_birth</code>
Cardinality	The numeric limit on a relationship: 1:1, 1:N, or M:N	Customer places many Orders -> 1:N
Participation Constraint	Whether involvement in a relationship is mandatory (total) or optional (partial)	Every Order needs a Customer (total); a Customer may have zero orders (partial)
Weak Entity	An entity that can’t be uniquely identified without its owner entity	Room identified only within a Building
Identifying Relationship	The relationship linking a weak entity to its owner	<code>belongs_to</code> between Room and Building

Term	Definition	Example
Partial Key (Discriminator)	The weak entity's own attribute(s), combined with the owner's key, for full identification	<code>room_number + building_id</code>
Recursive (Unary) Relationship	A relationship where an entity relates to another instance of itself	<code>Employee</code> manages <code>Employee</code>
Associative Entity	An entity introduced to resolve an M:N relationship, often holding relationship attributes	<code>Enrollment</code> between <code>Student</code> and <code>Course</code> , holding <code>grade</code>
Degree of a Relationship	Number of entity types involved: unary (1), binary (2), ternary (3)	<code>Supplier-Part-Project = ternary</code>

Structural Example

```

Building (1) ---- identifying relationship ----< (N) Room    [weak entity]
    PK: building_id                                partial key: room_number
                                                    full key: building_id + room_number

```

Common Confusions

Entity vs. Attribute The test is independence, not size. If it can stand alone with its own identity and could carry its own attributes -> entity. If it only describes something else -> attribute. Trap: `Author` as a text field on `Book` is an attribute — but if you need to track the author's birth date or other books, it needs to be promoted to its own entity.

Composite vs. Multivalued Attribute

- Composite = one value broken into sub-parts (one address -> street/city/zip).
- Multivalued = multiple independent values of the same kind (several phone numbers).
- A value can even be both: multivalued where each value is itself composite.

Cardinality vs. Participation Constraint

- Cardinality answers “how many?”
- Participation answers “is it mandatory?”
- They're independent — a 1:N relationship can still have an optional “1” side (e.g., a `Department` may have zero employees).

Weak Entity vs. Regular Entity with a Foreign Key Having a foreign key doesn't make an entity weak. The test is identification:

- Regular entity: has its own key that identifies it alone (`Order` has `order_id`, even though it references `customer_id`).
- Weak entity: genuinely cannot be told apart without the owner's key too (`Room` needs `building_id + room_number`).

Associative Entity vs. Weak Entity

- Weak entity exists because of an identification problem.
- Associative entity exists to resolve an M:N relationship (which can't be directly implemented relationally) and often to store attributes of the relationship itself. Trap: **Enrollment** (Student + Course + grade) is often mistaken for weak, but it's associative.

“Total” vs. “Complete”/“Full” Participation (a terminology slip from our practice) The correct standard term is total participation, not “complete” or “full” — same concept, but use “total” since that's what appears in textbooks and interviews.

Quick trick for participation: ask “can this entity exist without the relationship?” If no -> total. If yes -> partial.

Phase 2: Practice Log

Question 1 — Blog System (Difficulty: 1/10) (Solved)

Scenario: **Post** (**title**, **content**, **published_date**) and **Author** (**name**, **email**). One author writes many posts; each post has exactly one author.

Solution:

- Entities: **Post**, **Author**
- Relationship: **Author** writes **Post**
- Cardinality: 1:N
- Participation: **Post** = total (a post cannot exist without an author); **Author** = partial (an author can exist with zero posts)

Tip: The “can this exist without the relationship?” test resolves participation quickly and avoids the total/partial mix-up.

Question 2 — Online Bookstore (Difficulty: 3/10) — In Progress

Scenario: **Customer** (**name**, **email**) and **Book** (**title**, **price**, multiple **genres**). A customer can order many books; a book can be ordered by many customers. Each customer-book order needs **quantity** and **price_at_purchase** recorded.

(Answer pending — to be added once solved.)

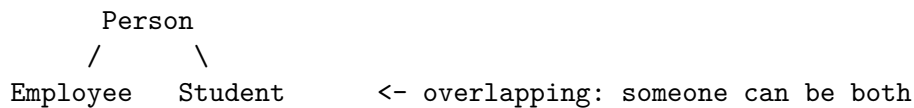
(NEW) Additional ER Modeling Concepts Not Yet Covered

These weren't part of our conversation yet, but are commonly tested alongside the above and worth knowing:

Generalization / Specialization (ISA Hierarchies)

A superclass entity can be broken into subclass entities that share the superclass's attributes but also have their own.

- Specialization — top-down: starting from a general entity and identifying more specific subtypes.
- Generalization — bottom-up: starting from specific entities and finding a common general entity.
- Disjoint constraint — an instance can belong to only ONE subclass (e.g., **Vehicle** -> **Car** OR **Truck**, never both).
- Overlapping constraint — an instance can belong to more than one subclass (e.g., **Person** -> **Employee** and **Student** simultaneously).
- Also has total (every superclass instance must belong to a subclass) vs partial specialization (some instances may belong to no subclass).



Aggregation

Lets you treat a relationship (plus its participating entities) as if it were a single higher-level entity, so it can itself participate in another relationship. Used when a relationship needs to relate to a third entity, but a plain ternary relationship doesn't fit the semantics. Example: (**Employee works_on Project**) treated as one unit, related to **Manager** via **monitored_by**.

Attribute Domain

The set of allowable values for an attribute — its data type plus any range or format constraints. Example: domain of **age** = positive integers, 0–120.

ER Diagram Notations

Two common conventions — worth recognizing since different tools/textbooks use different ones:

	Chen Notation	Crow's Foot Notation
Entity	Rectangle	Rectangle
Attribute	Ellipse (own shape)	Usually listed inside the entity box
Relationship	Diamond	Line between entities
Multivalued attribute	Double ellipse	N/A (shown as separate related entity)
Derived attribute	Dashed ellipse	N/A
Weak entity	Double-outlined rectangle	Rectangle with distinct marker
Cardinality	Numbers/letters near diamond	“Crow's foot” fork-shaped line-end symbols (e.g., a trident shape for “many”)

Crow's foot is the notation you'll most often see in real tools (MySQL Workbench, dbdiagram.io, Lucidchart); Chen notation is more common in textbooks for teaching the underlying concepts.

Ternary Relationships in Practice

Degree-3 relationships (e.g., **Supplier-Part-Project**) are trickier to express participation/cardinality for, since constraints can apply to combinations rather than single entities. In practice, they're often resolved into an associative entity connected to all three via binary relationships, similar to how M:N relationships are handled.

Recap

- Entity = independent “thing”; Attribute = describes an entity; Relationship = association between entities
- Attribute types: Key, Composite, Multivalued, Derived
- Cardinality (1:1 / 1:N / M:N) is separate from Participation (total / partial)
- Weak entities need an owner + identifying relationship + partial key to be fully identified
- Recursive/unary relationships connect an entity to itself; degree = number of entities involved (unary/binary/ternary)
- Associative entities resolve M:N relationships and often carry relationship attributes
- (NEW) Generalization/Specialization, Aggregation, Attribute Domains, and Chen vs. Crow's Foot notation round out the full ER modeling toolkit